PilotFish Sandbox — Interface Construction Playbook

Audience: Cursor agents (and humans reviewing agent work).
Scope: Any new or substantially changed PilotFish interface under `Clients/`.
Purpose: One repeatable way to design, scaffold, implement, document, and smoke-test interfaces so demos stay consistent and production risks are called out honestly.

Do **not** invent a one-off layout. Follow this playbook, then specialize only where the integration requires it.

How agents know how to build PilotFish interfaces

Cursor agents are **not** formally trained or certified by PilotFish. There is no special PilotFish fine-tune behind this Sandbox. Knowledge comes from a stack of sources when building an interface here:

1. Authoritative sources in this workspace

Priority	Source	Use it for
A	`PilotFish_Documentation/`	Module **inventory**, UI-type → class mapping, config semantics, deep-dive behavior (see §1.1)
A	`PilotFish_V2/` (XCS / module Java + `modules.conf`)	**Same EIP module code V1 uses** — FQCNs, `ConfigurationItem` names/defaults, listener/processor/transport behavior (see §1.2)
B	Working Sandbox demos under `Clients/**`	Proven **wiring** of those modules into V1 `route.xml` on `pilotfish-eip:23R1`
B	This playbook + `.cursor/rules/`	Construction process, V1 vs V2 policy, risks, definition of done
C	Your chat + `DESIGN.md`	Job-specific contracts and overrides
D	Live smoke tests / `logs/eip.log`	Confirm the chosen module loads on the demo image
E	General model knowledge	Only to fill gaps after A–D

Important: V2 does **not** invent a separate module stack. EIP modules (Listeners / Processors / Transports / Routing) are essentially the **same Java modules** whether the route document is classic V1 `route.xml` or V2 `route.v2.xml`. V2 mainly changes **how modules are connected and laid out**. Therefore agents **must use `PilotFish_V2` and `PilotFish_Documentation` to decide which modules exist and how they are configured**, then still **author runtime routes as V1** for this Sandbox's eiPlatform image unless the user overrides.

1.1 `PilotFish_Documentation/` (deep-dive library)

Path: `PilotFish_Documentation/`

Path	Role
`README.md`	Layout: Listeners / Processors / Transports × version (`XCS`, `23R1.127`, `26R1.11`, …)
`General/Process/PilotFish-Module-Documentation-Tracker.md` (+ PDF if present)	**Master inventory** — UI type, short class name, which versions have deep dives
`Listeners/`, `Processors/`, `Transports/` (`<version>/…`)	Deep-dive config reference for that module
`General/XCS/*`	Framework (threads, transactions) when designing concurrency / TX behavior
`General/Process/*`	How docs were generated — rarely needed for interface build

How to use when constructing an interface:

1. Open the **tracker** and find the UI type you need (e.g. Directory / File, HL7 LLP, Database Polling (SQL)).
2. Note the **class** (`DirectoryListener`, `DatabaseSqlListener`, …) and whether a deep-dive exists for a version close to the runtime image (`pilotfish-eip:23R1` demos → prefer `23R1.*` docs when present; otherwise use `26R1.11` / `XCS` and call out version skew in `DESIGN.md`).
3. Read the deep-dive for config item names, defaults, and pitfalls **before** inventing XML.
4. If the tracker lists a module but there is **no** deep-dive yet, fall through to `PilotFish_V2` Java / `modules.conf` (§1.2), then a Sandbox example if any.

Do **not** skip Documentation when a relevant deep-dive exists.

1.2 `PilotFish_V2/` (module source + catalog — shared with V1)

Primary tree: `PilotFish_V2/pilotfishdevelopment-xcs-b313c66ccd5f/`

Path	Role
`XCS-console-gui/.../modules.conf`	**Catalog of loadable module FQCNs**
`subprojects/modules-*/src/main/java/com/pilotfish/eip/modules/**`	Module implementation — package + class = `route.xml` `class="…"`
Configuration / descriptor types on those classes	Exact **config element names** and defaults under `<Configuration>`
`.clinerules/BLUEPRINT.md` (or `.aiassistant/rules/BLUEPRINT.md`)	V1 stage model vs V2 node model; what conversion should preserve
`V2_RouteViewer/`	Viewer/reference assets for diagrams

How to use when constructing an interface:

1. Search `modules.conf` and/or Java under `com/pilotfish/eip/modules` for the capability (e.g. `SNIP`, `DirectoryListener`, `HL7TCP`).
2. Resolve the **fully qualified class name** from the Java `package` + class (or the `modules.conf` line).
3. Read configuration item definitions (and nearby tests/resources) so XML tags match the code — **do not invent tag names**.
4. Prefer modules that also appear in Sandbox demos when you need a drop-in XML skeleton; otherwise scaffold from V2 + Documentation, then smoke-test on `pilotfish-eip:23R1`.
5. Use BLUEPRINT / importer notes when mapping V1 Source/Target/Router into generated `route.v2.xml`.

Wrong package / class strings have already broken demos. Treat FQCNs as **looked-up constants** from Documentation/V2 (or a working demo), never as creative writing.

1.3 Sandbox demos (`Clients/**`) — proven assemblies

Best source for **end-to-end patterns** that already run in Docker:

Demo	Pattern
`Clients/Demos/edi-837-snip-sqlserver/`	SQL poll → EIP handoff → fork → EDI + SNIP
`Clients/Demos/hl7-healthcare-automation/`	Directory → validate → router fan-out → SQL + file

Default assembly method: choose modules via Documentation + V2 → copy a Sandbox `route.xml` fragment when one exists for that class → otherwise build V1 XML from looked-up config items → convert to V2 for diagrams.

2. How agents pick processors / class names (`route.xml`)

Decision order:

1. **Identify the capability** (directory listen, SQL poll, XSLT, SNIP, HL7 LLP, …).

2. **`PilotFish_Documentation` tracker** → UI type + class; read deep-dive if present.
3. **`PilotFish_V2` `modules.conf` + Java** → confirm FQCN and configuration item XML names.
4. **Sandbox demo `route.xml`** → copy a working module block when available.
5. **Smoke test** on the demo EIP image; fix using logs if the class is missing from that WAR (docs/V2 may be newer than `23R1`).
6. **Ask the user** if the module isn't in the image and can't be substituted.
7. **General model knowledge** only as a last-resort hint — never the source of an FQCN.

3. V1 vs V2 — runtime vs diagrams

Artifact	Format	Role
`pilotfish/demo-eip-root/**/route.xml`	**V1**	**Runtime** for `pilotfish-eip:23R1` — **author here**
`eip-root/**/route.v2.xml` + `modules/*.xml`	**V2**	Routes tab / docs / PDF — usually **generated** from V1
`tools/convert_routes_to_v2.py`	V1 → V2	Keeps `route.xml`; emits diagram graph
`PilotFish_V2/`	Product + **shared module source**	Module catalog & behavior **and** V2 format/editor understanding
`PilotFish_Documentation/`	Deep dives + tracker	Interface creation decisions (which module, which settings)

Policy: Use V2 **code** and Documentation to choose and configure modules; still **ship V1 `route.xml` as runtime** and generate V2 for visualization, unless the user explicitly wants native V2 runtime.

4. Instructions you give in chat

Your prompts, corrections, ports, case-study links, and overrides win over defaults. `DESIGN.md` is the working spec for that interface.

5. General model knowledge (not PilotFish training)

Horizontal knowledge (integration patterns, HL7/EDI concepts, Docker, SQL, Flask) — **not** a substitute for `PilotFish_Documentation` or `PilotFish_V2` when picking modules.

6. External / fetched material (when used)

Public case studies and links you provide; live smoke evidence. Prefer in-repo Documentation + V2 over stale web pages when they conflict on config item names.

7. What is *not* a source of authority

- Invented FQCNs or config element names

- “Only demos may be used” as a hard ceiling when Documentation/V2 clearly provide a better module (demo copying is preferred for **skeletons**, not an excuse to ignore the catalog)
- Claiming compliance without fail-closed config
- Assuming Documentation 26R1 deep-dives always match `pilotfish-eip:23R1` without smoke test
- Hand-maintained V2 as the only runtime truth for current Sandbox demos

Practical rule of thumb

```
Your chat + DESIGN.md
    → PilotFish_Documentation (tracker + deep dives)
        → PilotFish_V2 (modules.conf + module Java = same modules as V1)
            → nearest Sandbox demo route.xml (assembly skeleton)
                → author V1 runtime → convert to V2 for viewer/PDF
                    → smoke test on pilotfish-eip:23R1
```

If those disagree, **prefer V2/docs for module identity + Sandbox/live smoke for “loads on this image”**, and document version skew in `DESIGN.md`.

---

0. When this applies

Use this playbook when asked to:

- Create a new PilotFish demo / interface / route stack
- Port a case study into a runnable Sandbox demo
- Add a route, listener, transform, or transport to an existing Sandbox interface
- Produce a design document for an interface

If the task is a tiny fix (typo, port conflict, one SQL row), skip the full scaffold and only apply relevant sections (smoke test, honesty about validation, no secret sprawl).

---

1. Pre-flight (block implementation until answered)

Capture answers in the interface `DESIGN.md` (see §8). If missing, ask the user.

#	Question	Why
1	**Business job** in one sentence?	Keeps scope honest
2	**Inbound contract** (format, envelope, transport: dir / MLLP / SQL / API)?	Wrong listener = silent no-ops
3	**Outbound contract(s)** (file / SQL / EIP / HTTP) and **success criteria**?	Dual-write needs an explicit model
4	**Validation** level (none / heuristic / SNIP / HL7 structural / partner ACK)?	Don't claim compliance without gates
5	**Identity & idempotency** keys?	Dedup / retries
6	**Happy-path sample(s)** available?	Seed data drives the first green path
7	**Demo vs production labeling** — is this demo-only?	Secrets, SA, PHI mounts
8	**Ports** free on this machine?	Avoid collisions with other Sandbox demos

Port allocation (default pattern): pick an unused triple near existing demos:

Role	Existing examples
SQL Server host	14335 (EDI), 14336 (HL7)
EIP host	8093 (EDI), 8096 (HL7)
Web UI host	8095 (EDI), 8097 (HL7)

Document chosen ports in `README.md` before `compose up`.

---

2. Standard directory scaffold

Create under `Clients/Demos/<slug>/` (or `Clients/<Client>/<slug>/` for client work):

```
<interface>/
  README.md
  DESIGN.md                       # filled from §8 template
  docker-compose.yml
  sql/
    01_init.sql                   # idempotent preferred; document if destructive
  samples/                        # inbound fixtures (named clearly)
  input/                          # runtime drop dirs (gitkeep / .gitkeep)
  output/                         # artifacts (edi, snip, clearinghouse, kickout, archive…)
  eip-root/                       # diagram / V2 source of truth for route viewer
    interfaces/<App>/routes/<N - Name>/
      route.xml
      route.v2.xml                # generate/convert; don't hand-edit unless required
      modules/*.xml
  pilotfish/
    demo-eip-root/                # what the container actually mounts at runtime
    Dockerfile                    # FROM pilotfish-eip:23R1 (+ JDBC if needed)
    demo-entrypoint.sh
    conf/environment-settings.conf
  webui/                          # Flask (or existing demo UI pattern)
    app.py
    templates/
    static/
      route-viewer/               # copy from an existing demo; keep in sync
  tools/
    convert_routes_to_v2.py       # if needed
    export_route_diagrams.py      # PDF export with --config → documents/
  documents/                      # required deliverable: route design PDF(s)
    <Interface>_V2_Route_Diagrams.pdf
  logs/                           # gitignored runtime logs if bind-mounted
```

Source of truth: runtime behavior = `pilotfish/demo-eip-root`.
Diagrams / Routes tab = `eip-root` (or a documented sync job).
Route design PDF = `documents/` (generated; do not leave only under `output/route-diagrams`).
If runtime and diagrams diverge, fix or document the divergence in `DESIGN.md` — never leave silent drift.

Reference implementations:

- `Clients/Demos/edi-837-snip-sqlserver/` — SQL poll → EIP handoff → fork → EDI + SNIP
- `Clients/Demos/hl7-healthcare-automation/` — directory listen → validate → router fan-out → SQL + file

---

3. Route construction pattern

3.1 Canonical stage chain (V1 runtime)

Author this chain in **V1** `route.xml` (what `pilotfish-eip:23R1` runs):

```
Listener
  → Source processors (normalize / validate / map / snapshot files)
  → FormatProfile (relay | XPath fork when true multi-message)
  → XPathRoutingModule (Conditional Router)
  → Target processors (pre-transport map)
  → Transport(s)   # Directory | DatabaseSql | EIP | HTTP …
```

Then run `tools/convert_routes_to_v2.py` (or demo equivalent) so `route.v2.xml` stays aligned for the Routes tab / PDF. Do **not** hand-maintain V2 as the only copy of runtime truth.

3.2 Naming

- Routes: `1 – <Verb Object>`, `2 – …` (same spirit as existing demos)
- Modules: stable UUIDs once created; don't regenerate casually (breaks V2 links)
- Env props: `$$sqlserver.*`, `$$*_DIRECTORY` in `environment-settings.conf`

3.3 Must-design decisions (write into DESIGN.md)

1. **When is business state committed?**
   Never mark work terminal (`PROCESSED` / archived-as-success) before all required side effects succeed — or document that the demo deliberately does claim-before-complete and list it under Risks.

2. **Dual / multi write:**
   If router fans out to SQL **and** files (or multiple files), define order + compensation (outbox, or "file then SQL", or accept demo risk explicitly).

3. **Fork vs snapshot:**
   A stage named "Split" / "Batch" must either truly fork (`XPathForkingModule` / equivalent) or be renamed (e.g. "Batch Flag Snapshot"). No fake splits.

4. **Validation gate:**
   If SNIP / HL7 / XSLT "validation" runs with `StopOnError=false` and always continues, say so. Prefer fail-closed kickout + non-success status for anything marketed as validation.

5. **Validate the wire artifact:**
   Don't SNIP / schema-validate a pre-transform or stripped XML if the partner receives the post-transform payload.

6. **Batching / back-pressure:**
   SQL listeners: bounded `TOP (@n)` + locking hints when claiming rows.
   Triggerable / forked routes: throttling when heavy processors (SNIP) sit downstream.

7. **Idempotency:**
   Unique business keys in SQL; versioned or non-overwrite filenames for audit outputs.

8. **Poison / kickout:**
   Explicit fail directory or status; don't Move-to-archive on exception-only paths.

---

4. Implementation order (agent checklist)

Work in this order unless the user specifies otherwise:

1. **DESIGN.md** filled from §8 (can be draft; must exist before claiming "done").
2. **Module selection** via `PilotFish_Documentation` tracker/deep-dives + `PilotFish_V2` (`modules.conf` / module Java). Record chosen FQCNs in DESIGN.md Pipeline table.
3. **SQL init + samples** (if applicable).
4. **environment-settings.conf + compose + Dockerfile** (ports, volumes, heap if SNIP).
5. **Minimal Route 1** V1 `route.xml` (listener → snapshot/file) green — copy Sandbox skeleton when possible.
6. **Downstream routes / transforms** one stage at a time (still V1 runtime).
7. **Router + transports** with kickout path tested once.
8. **Web UI** inject/submit + status views matching the demo story.
9. **V2 convert** into `eip-root` + Routes tab / docs.
0. **Route design PDF (required):** with Web UI up, run `python3 tools/export_route_diagrams.py --config changed` and write/copy the PDF into **`documents/`** at the interface root (see §6).
1. **README.md** (run, ports, smoke commands; link to `documents/*.pdf`).
2. **Smoke test** (§7) and paste results into the chat (and update DESIGN.md Risks if findings).

Do not refactor unrelated demos. Prefer copying the closest existing demo assembly after modules are chosen from Documentation/V2.

---

5. Runtime / Docker conventions

- Base image: `pilotfish-eip:23R1` (build from Sandbox root if missing).
- Bind-mount runtime interfaces and `input/` / `output/` / `logs/` as other demos do.
- Heap: if SNIP (or similarly heavy) processors load, set sufficient `CATALINA_OPTS` (EDI demo uses ~6GB max) and document why.
- Web UI `depends_on` EIP when useful; document cold-start wait (often 60–90s).

- Secrets: demo password may match existing demos for local convenience, but **DESIGN.md and README must say demo-only** — never invent "production-ready" claims around `sa` + shared passwords.
- Prefer least-privilege DB users when creating new non-demo client interfaces.

---

6. Web UI, route viewer, and PDF

When the interface has routes (every new Sandbox interface with a route stack):

1. Copy `webui/static/route-viewer/` from a current demo (keep layout + Box config modes).
2. Serve `route.v2.xml` + module XML via `/api/v2/routes…` (mirror existing Flask helpers).
3. Routes tab: route select, layout dropdown, link to docs.
4. Docs / print view and `tools/export_route_diagrams.py` must use the same **Box config** modes:
   - `compact` — names only
   - `changed` — non-default values (**default for docs/PDF**)
   - `all` — all ModuleConfig values
5. Export script: `python3 tools/export_route_diagrams.py --config changed`
   Screenshot URL must include `mode=docs&bare=1&config=<mode>`.
6. PDF and on-screen docs view should show equivalent config detail (same default).

6.1 Route design PDF → `documents/` (required for every new interface)

Any time you **create a new interface** (or add/change routes enough that diagrams should be refreshed), you **must** generate the route design PDF automatically as part of finishing the work — do not wait for the user to ask.

1. Ensure Web UI is up and V2 routes render.
2. Run `python3 tools/export_route_diagrams.py --config changed` (default Box config).
3. Place the final PDF under the interface's **`documents/`** folder, e.g.:
   - `documents/<ShortName>_V2_Route_Diagrams.pdf`
   - Also keep or symlink `…_changed.pdf` if the exporter writes a mode suffix.
4. Point `export_route_diagrams.py` output at `documents/` **or** copy/move from `output/route-diagrams/` into `documents/` before declaring done.
5. Mention the PDF path in `README.md`.

Skipping the PDF requires the user to opt out explicitly; "optional" is not the default.

---

7. Smoke test (required before "done")

Minimum bar:

- [ ] `docker compose up -d --build` comes healthy (or known wait documented)
- [ ] One happy-path sample produces **every** expected side effect (SQL row and/or files)

- [ ] Kickout / fail path exercised once **or** explicitly marked “not implemented” in DESIGN.md
- [ ] Routes tab renders `route.v2.xml`
- [ ] **Route design PDF** written under `documents/` (`--config changed`)
- [ ] No silent claim of partner-grade validation unless gated

Record: ports, sample used, output paths, and any known failures (e.g. SNIP noise).

---

8. DESIGN.md template (copy into each interface)

```
# <Interface name> — Design

## 1. Purpose
<one paragraph>

## 2. Context / actors
- Sources:
- Destinations:
- Demo vs production:

## 3. Inbound contract
- Transport:
- Format / envelope:
- Identity fields:
- Samples path:

## 4. Outbound contract(s)
| Destination | Format | Success criterion |
|-------------|--------|-------------------|
|             |        |                   |

## 5. Pipeline
| Stage | Module / mechanism | Notes |
|-------|--------------------|-------|
| Listener | | |
| Processors | | |
| Router | | |
| Transports | | |

## 6. State & idempotency
- Status model:
- When state advances:
- Dedup keys:
- Retry / poison:

## 7. Validation
- What is checked:
- What is NOT checked:
- Does failure block outbound? (yes/no):

## 8. Dual-write / side effects
- Order of commits:
- Compensation:
- Demo shortcuts (if any):

## 9. Risks & bottlenecks
| Severity | Risk | Why it bites here | Mitigation / accepted? |
```

```
|----------|------|------------------|----------------------|
| High | | | |
| Med | | | |
| Low | | | |

## 10. Ops
- Ports:
- Volumes:
- Heap / special JVM:
- Dependencies / cold start:

## 11. Observability
- Logs:
- Kickout dir:
- Transaction / debug tracing: on/off and retention:

## 12. Open questions
-
```

Risks checklist (seed §9 from these when applicable)

- Claim-before-complete (status flip before all side effects)
- Dual-write without outbox / compensation
- Validation theater (runs but doesn't gate; wrong artifact validated)
- Fake batch split / unused multi-source folders
- Unbounded poll / no throttling into heavy stages
- Overwrite semantics on audit files
- Missing unique keys → duplicates on resubmit
- Runtime vs `eip-root` config drift
- Secrets / PHI on permissive mounts
- Partner transport missing (e.g. no MLLP/ACK when story implies it)

---

9. README.md minimum

Every interface README must include:

1. What it does (bullet flow)
2. Prerequisites (`pilotfish-eip:23R1`, Docker)
3. `docker compose up` instructions
4. Ports table
5. Smoke / useful commands
6. Explicit **Demo only** callouts for credentials / PHI / validation limits

---

10. Anti-patterns (do not ship without calling out)

Anti-pattern	Do instead
Name a stage “Validate” / “SNIP” / “Split” that doesn’t gate or fork	Rename or implement the real behavior
`UPDATE … PROCESSED` in the same SQL that claims work	Claim → process → terminal status
Router fan-out with `retries=1` and no reconcile	Outbox or document as demo risk
Listener `FileExtensionRestriction` mismatched to story samples	Align contract + samples + UI
Hand-maintained duplicate route trees that disagree	One generator direction; document sync
Claiming “HL7 automation” / “SNIP compliant” from heuristic passes	Accurate language in README + DESIGN
World-writable PHI without demo labeling	Label + restrict for client work
Skipping DESIGN.md because “it’s just a demo”	Short DESIGN.md still required

---

11. Definition of done

An interface construct is done when:

1. DESIGN.md exists and §9 Risks is filled (accepted demo risks allowed if labeled).
2. Compose stack runs and smoke test passes.
3. README is enough for a cold start on this machine.
4. Runtime config and diagrams agree **or** drift is documented.
5. Web/route viewer present when routes are part of the deliverable.
6. **Route design PDF** exists under `documents/` (generated with `--config changed`).
7. Agent summary lists known limitations in plain language (no compliance theater).

---

12. Quick command pack (adapt ports/paths)

```
cd "Clients/Demos/<slug>"
docker compose up -d --build
docker compose logs -f pilotfish
# happy path: drop sample or use Web UI
ls -la output/*
python3 tools/export_route_diagrams.py --config changed
# ensure PDF is under documents/ (exporter may write output/route-diagrams/ — copy if needed)
mkdir -p documents && cp -f output/route-diagrams/*Route_Diagrams*.pdf documents/ 2>/dev/null ||
        true
ls -la documents/
docker compose down -v   # destroys DB volume — warn user first
```